Laurea Triennale in informatica-Università di Salerno Corso di Ingegneria del Software- Prof.ssa F. Ferrucci, Prof. F. Palomba

Test Plan

C-09 SINTONIA

Riferimento	2025_C09_TP
Versione	1.0
Data	23/11/2025
Destinatario	Prof.ssa Filomena Ferrucci, Prof.re Fabio Palomba
Presentato da	C09 Team Sintonia
Approvato da	//

SINTONIA
La mente trova pace, un passo alla volta

Revision History

Data	Versione	Descrizione	Autori
23/11/2025	1.0	Configurazione Documento	Alessio Del Sorbo
23/11/2025	1.0	Aggiunte features da testare/non testare	Francesco Coppola
23/11/2025	1.0	Aggiunta Panoramica del sistema	Mattia Fanzini
23/11/2025	1.0	Aggiunto Materiale di Testing	Gianni Policola

SINTONIA
La mente trova pace, un passo alla volta

Team Members

Ruolo	Nome e Cognome	Acronimo	Contatto
PM	Gianfranco Barba	GB	g.barba14@studenti.unisa.it
PM	Francesco Corcione	FC	f.corcione5@studenti.unisa.it
TM	Gianluca Ambrosio	GA	g.ambrosio39@studenti.unisa.it
TM	Elena Carlomagno	EC	e.carlomagno1@studenti.unisa.it
TM	Rosaria Cervino	RC	r.cervino3@studenti.unisa.it
TM	Francesco Coppola	FCP	f.coppola99@studenti.unisa.it
TM	Antonio Walter De Fusco	ADF	a.defusco1@studenti.unisa.it
TM	Gavino De Stefano	GDS	g.destefano41@studenti.unisa.it
TM	Alessio Del Sorbo	ADS	a.delsorbo32@studenti.unisa.it
TM	Francesco Di Giovanni	FDG	f.digiovanni10@studenti.unisa.it
TM	Mattia Fanzini	MF	m.fanzini@studenti.unisa.it
TM	Angelo Fusco	AF	a.fusco58@studenti.unisa.it
TM	Gianni Policola	GP	g.policola@studenti.unisa.it

SINTONIA
La mente trova pace, un passo alla volta

Sommario

Sommario 4
1 Introduzione 6
2 Relazione con altri documenti 7
3 Panoramica del Sistema 8
4 Feature da testare / da non testare 9
4.1 Features da testare (Features to be tested) 10
4.2 Features da non testare (Features not to be tested) 11
5 Pass / Fail Criteria 12
6 Approccio 13
6.1 Testing di Unità 13
7 Materiale di Testing 14
8 Test Cases 15
1 Compilazione questionario (UC_PZT_3) 15
1.1 Unit 1: Validazione Compilazione Questionario (service) 15
1.2 Unit 2: Esecuzione Recupero Domande (service) 17
1.3 Unit 3: Validazione Calcolo Score (service) 19
1.4 Unit 4: Esecuzione Calcolo Score (service) 21
1.5 Unit 5: Validazione Invio (service) 22
1.6 Unit 6: Esecuzione Invio (service) 24
1.7 Unit 7: Gestione Richiesta Get (controller) 25
1.8 Unit 8: Gestione Richiesta Start (controller) 26
1.9 Unit 9: Gestione richiesta Submit (controller) 27
2 Inserimento Stato d'Animo (UC_PZT_5) 28
2.1 Unit 1: Validazione Dati (service) 28
2.2 Unit 2: Esecuzione Creazione (service) 32
2.3 Unit 3: Gestione richiesta (controller) 34
3 Inserimento Contenuto Diario Personale (UC_PZT_19) 36
3.1 Unit 1: Validazione Contenuto Diario (service) 36
3.2 Unit 2: creazione pagina diario (service) 38
3.3 Unit 3: Gestione richiesta (control) 40
4 Inserimento Domanda Forum (UC_PZT_23) 43
4.1 Unit 1: Validazione Domanda (service) 43
4.2 Unit 2: Inserimento Domanda (service) 46
4.3 Unit 3: Inserimento Domanda (control) 48
5 Revisione questionario paziente (UC_PSI_1) 50
5.1 Unit 1: Validazione revisione (service) 50
5.2 Unit 2: Esecuzione Revisione Questionario (service) 54
5.3 Unit 3: Gestione richiesta (controller) 55

5 Revisione questionario paziente (UC_PSI_1) 57
5.1 Unit 1: Validazione revisione (service) 57
5.2 Unit 2: Esecuzione Revisione Questionario (service) 61
5.3 Unit 3: Gestione richiesta (controller) 62
6 Visualizzazione Alert Clinico (UC_PSI_3) 64
6.1 Unit 1: Esecuzione Visualizzazione Alert(service) 64
6.2 Unit 2: Gestione visualizzazione (controller) 65
7 Terminazione Cura Paziente(UC_PSI_4) 67
7.1 Unit 1: Validazione terminazione cura (service) 67
7.2 Unit 2: Esecuzione terminazione cura (service) 69
7.3 Unit 3: Gestione richiesta (controller) 71
8 Generazione Report(UC_PSI_5) 73
8.1 Unit 1: Gestione richiesta (service) 73
8.2 Unit 2: Esecuzione Generazione Report (service) 74
8.3 Unit 3: Gestione richiesta (controller) 76
9 Modifica Priorità Paziente(RF_AS_10) 78
9.1 Unit 1: Validazione nuova priorità (service) 78
9.2 Unit 2: Esecuzione modifica priorità (service) 80
9.3 Unit 3: Gestione richiesta (controller) 81
10 Modifica Assegnazione Psicologo (UC_AS_11) 83
10.1 Unit 1: Validazione assegnazione (service) 83
10.2 Unit 2: Esecuzione assegnazione (service) 85
10.3 Unit 3: Gestione richiesta (controller) 87
11 Accettazione richiesta invalidazione (UC_AS_21) 89
11.1 Unit 1: Validazione richiesta (service) 89
11.2 Unit 2: Ricerca precedente questionario con cambiamento (service) 93
11.3 Unit 3: Ricalcolo score (service) 96
11.4 Unit 4: Downgrade priorità (service) 99
11.5 Unit 5: Esecuzione Invalidazione Questionario (service) 102
11.6 Unit 6: Gestione richiesta (controller) 103

SINTONIA
La mente trova pace, un passo alla volta

1 Introduzione

SINTONIA è una piattaforma che ha come obiettivo quello di ottimizzare la gestione delle liste d'attesa per il supporto psicologico nella sanità pubblica, fornendo gli strumenti necessari per un triage dinamico e un monitoraggio proattivo del paziente tramite un ambiente centralizzato in cui psicologi e amministratori possano intervenire tempestivamente.

In tale contesto, il presente documento costituisce il **Piano di Test (Test Plan)** formale per il progetto e si pone come strumento fondamentale per pianificare, monitorare e controllare l'intero processo di verifica e validazione del software. La stesura di questo elaborato nasce dall'esigenza di definire una strategia strutturata che permetta di identificare con chiarezza il perimetro operativo delle attività di collaudo.

Nel prosieguo del documento verrà infatti delineato lo scope specifico della fase di test, distinguendo rigorosamente le funzionalità oggetto di verifica da quelle escluse, e verranno formalizzati i criteri oggettivi di accettazione e rifiuto (Pass/Fail Criteria) necessari per valutare l'esito delle prove. La trattazione approfondirà inoltre l'approccio metodologico adottato e le risorse strumentali richieste, per culminare nella definizione puntuale dei casi di test (Test Cases), i quali rappresentano la declinazione pratica delle specifiche funzionali in procedure di verifica eseguibili.

2 Relazione con altri documenti

Il presente Test Plan è strettamente correlato ai documenti prodotti per il progetto SINTONIA e verrà aggiornato nelle successive iterazioni qualora tali documenti subiscano variazioni o vengano rilasciati ulteriori artefatti progettuali. I test case definiti in questo piano sono derivati e organizzati in coerenza con:

- **Relazione con il Requirements Analysis Document (RAD).**
  I test case pianificati sono costruiti a partire dai requisiti funzionali e non funzionali e dagli use case descritti nel RAD. Ogni caso di test è quindi tracciabile rispetto ai requisiti che intende verificare, secondo un approccio requirements-based.

- **Relazione con il System Design Document (SDD).**
  I test case rispettano la decomposizione in sottosistemi, l'architettura a tre livelli e le interfacce dei servizi definite nell'SDD. La struttura del testing è organizzata in modo consistente con tali scelte progettuali.

3 Panoramica del Sistema

Il sistema proposto si basa su un'architettura di tipo **Three-Tier** (a tre livelli), la quale garantisce una netta separazione delle responsabilità tra presentazione, logica applicativa e dati, facilitando la manutenibilità del software.

Durante lo sviluppo dell'interfaccia utente verrà utilizzata la libreria **React**. Questa scelta è motivata dalla sua architettura a componenti (**Component-Based Architecture**) che permette il riutilizzo del codice e dall'utilizzo del **Virtual DOM**, che garantisce elevata efficienza e velocità di rendering.

Per la parte di front-end e la generazione delle view verranno usati:

- HTML5
- CSS3
- JavaScript (React)

Per quanto riguarda invece la logica applicativa di back-end sarà utilizzato il framework **NestJS**. Costruito su **NodeJS** e **TypeScript**, esso è stato selezionato per la sua architettura modulare e scalabile, oltre che per la tipizzazione forte che riduce gli errori in fase di compilazione.

Per la gestione della persistenza dei dati verrà utilizzato il DBMS relazionale **PostgreSQL** , una soluzione scelta per la sua affidabilità, conformità agli standard SQL e capacità di gestire strutture dati complesse in modo sicuro.

4 Feature da testare / da non testare

Il presente capitolo definisce il perimetro operativo (*test scope*) delle attività di verifica e validazione per l'attuale fase di rilascio. La strategia di testing adottata è di tipo *requirements-based*: l'obiettivo è garantire la completa conformità del sistema rispetto alle specifiche funzionali concordate. Di conseguenza, il criterio di inclusione per le attività di testing è la **copertura integrale dei Casi d'Uso (Use Cases)** descritti nel Documento di Analisi dei Requisiti (RAD).

Nota specifica per il modulo di reportistica (SintonIA): La funzionalità relativa alla generazione report mediante il modulo di SintonIA sarà esposta a ulteriori test a causa della sua complessità e della natura specifica degli algoritmi coinvolti, tale modulo sarà oggetto di una fase di validazione che impiegherà tecniche di testing dedicate (es. validazione statistica o test di accuratezza algoritmica). La relativa pianificazione e i risultati saranno dettagliati in una documentazione di test specifica e distinta dal presente documento.

4.1 Features da testare (Features to be tested)

Saranno oggetto di verifica puntuale tutte le funzionalità che costituiscono la realizzazione operativa degli Use Case documentati nel RAD. Nello specifico, saranno testati i seguenti requisiti funzionali associati agli attori del sistema:

- **Lato paziente (PZT):**
  - **[RF_PZT_4]** Compilazione Questionario.
  - **[RF_PZT_5]** Inserimento stato d'animo.
  - **[RF_PZT_19]** Inserimento contenuto diario personale.
  - **[RF_PZT_23]** Inserimento domanda forum.
- **Lato psicologo (PSI):**
  - **[RF_PSI_1]** Revisione questionario paziente.
  - **[RF_PSI_3]** Visualizzazione sezione alert clinici.
  - **[RF_PSI_5]** Generazione report.
  - **[RF_PSI_7]** Terminazione cura paziente.
- **Lato amministratore di sistema (AS):**
  - **[RF_AS_10]** Modifica priorità paziente.
  - **[RF_AS_11]** Modifica assegnazione psicologo.
  - **[RF_AS_17]** Accettazione richiesta invalidazione.

4.2 Features da non testare (Features not to be tested)

Le seguenti funzionalità non verranno verificate in questa fase:

- **Funzionalità a priorità bassa:** I requisiti funzionali non elencati nella sezione 4.1 sono considerati a priorità bassa e sono esclusi dallo scope di questa sessione di test, pertanto non saranno verificati in questa iterazione.

5 Pass / Fail Criteria

Le attività di testing sono mirate a identificare la presenza di errori nel sistema.

Bisognerà testare uno use case per ogni membro del team.

Criteri di Successo (Pass): Un caso di test viene considerato superato se il comportamento osservato nel sistema SINTONIA coincide perfettamente con il risultato atteso descritto nel caso d'uso. Nello specifico, per ogni funzionalità testata, devono verificarsi le seguenti condizioni:

- L'interfaccia utente fornisce un feedback visivo di conferma.
- Eventuali notifiche o comunicazioni previste dal flusso vengono generate e inviate correttamente.

Criteri di Fallimento (Fail): Un caso di test è considerato fallito se si verifica una qualsiasi delle seguenti anomalie:

- Il risultato attuale differisce da quello atteso.
- Il sistema non fornisce alcun feedback all'utente o mostra messaggi di errore non previsti.
- Le funzionalità di sicurezza vengono aggirate.
- Il sistema entra in uno stato inconsistente o si blocca durante l'esecuzione.

6 Approccio

I casi di test saranno definiti utilizzando prevalentemente un approccio Black-Box , focalizzandosi sul comportamento atteso (input/output) definito nelle interfacce dei servizi. Tuttavia, per garantire la robustezza, verrà monitorata la copertura del codice (branch coverage) puntando a raggiungere una percentuale dell'80% sufficiente per garantire che servizi più complessi come la gestione del triage dinamico siano testati con un dettaglio significativo.

6.1 Testing di Unità

Per il testing di unità del sistema SINTONIA, la strategia prevede di testare singolarmente i metodi delle classi appartenenti ai layer di Controllo (Controller) e Logica Applicativa (Service), come definiti nel System Design Document, riguardanti gli use case sviluppati nel Requirements Analysis Document.

- **Cosa testare**: Verranno sottoposti a test tutti i metodi che implementano logica di business o di validazione coinvolti nel flusso di operazione per la riuscita di un determinato use case.
- **Cosa escludere**: Saranno escluse dal testing unitario le classi DAO e le interfacce pure, in quanto prive di logica comportamentale complessa dato che conterranno esclusivamente metodi di accesso ai dati come getter e setter.

Considerando che il backend di SINTONIA è sviluppato in **NestJS**, gli strumenti scelti per il testing unitario saranno:

- **Framework**: Jest sarà il framework principale utilizzato per la scrittura e l'esecuzione dei test unitari. Jest è lo standard per l'ecosistema NestJS/TypeScript e offre funzionalità integrate per le asserzioni e il running dei test.
- **Mocking:** Poiché i Service di Sintonia dipendono dai DAO per l'accesso al database PostgreSQL , e i Controller dipendono dai Service, è necessario isolare l'unità sotto test. Verranno utilizzati oggetti Mock (tramite le funzionalità di mocking di Jest) per simulare il comportamento delle dipendenze senza accedervi realmente.

7 Materiale di Testing

Le risorse che verranno utilizzate per l'esecuzione del testing si compongono di materiale redatto in precedenza, quindi hardware e risorse software.

Per quanto concerne il materiale redatto, le risorse utilizzate come riferimento per identificare i requisiti e le componenti da testare saranno:

- **Requirements Analysis Document (RAD):** per l'identificazione degli Use Case e dei requisiti funzionali e non funzionali.
- **System Design Document (SDD):** per il rispetto dell'architettura Three-Tier e la verifica dei servizi dei sottosistemi.
- **Design Pattern Document (DPD):** per il testing delle logiche specifiche implementate tramite pattern (es. Adapter per il modulo IA).

Relativamente alla **componente hardware**, è necessario il semplice utilizzo di un computer capace di eseguire l'ambiente di runtime Node.js e i browser web per la simulazione lato client.

Invece, a proposito dei **tools software**, verranno utilizzati strumenti coerenti con lo stack tecnologico e con la strategia di testing di unità:

- **Framework di Testing Unitario:** Verrà utilizzato **Jest**, standard per l'ecosistema NestJS/TypeScript. Jest svolgerà sia il ruolo di test runner sia quello di libreria di asserzioni per verificare singolarmente i metodi delle classi Controller e Service.
- **Strumenti di Mocking**: Verranno utilizzate le funzionalità integrate di **Jest** per la creazione di oggetti Mock. Questi sono essenziali per isolare l'unità sotto test, simulando il comportamento delle dipendenze senza accedervi realmente durante i test di unità.
- **Ambiente di Esecuzione**: Saranno necessari **Node.js** come ambiente runtime, **Nest.js** come framework applicativo e **npm** per la gestione delle dipendenze e per eseguire i test.
- **Gestione Dati**: Un'istanza di **PostgreSQL** sarà necessaria per i test di integrazione e di sistema, sebbene esclusa dal testing unitario puro tramite mocking.

8 Test Cases

1 Compilazione questionario (UC_PZT_3)

1.1 Unit 1: Validazione Compilazione Questionario (service)

Obiettivo: Verificare la validità dell'input per il recupero del questionario.

Parametro:	ID Questionario
Nome categoria:	Formattazione

Validazione [VALQ]	None
	`1. ID Questionario Null o Vuoto = false [error].`
	`2. ID Questionario recuperato = true [PROPERTY VALUE OK].`

Parametro:	Tipologia
Nome categoria:	Presenza ed Esistenza

Validazione [VALT]	None
	`1. Tipologia non trovata = false [error].`
	`2. Tipologia valida ed esistente = true [PROPERTY VALT_OK].`

SINTONIA
La mente trova pace, un passo alla volta

Test Case ID	Test frame	Esito
TC RF 3_1	VAL Q1	**Errato**: ID Questionario vuoto.
TC_RF3_2	VAL Q2,VALT 1	**Errato**: Tipologia non trovata.
TC RF 3_3	VAL Q2, VALT 2	**Corretto**: Questionario e Tipologia validi.

SINTONIA
La mente trova pace, un passo alla volta

1.2 Unit 2: Esecuzione Recupero Domande (service)

Obiettivo: Verificare la ricerca nel DB e il parsing delle domande.

Parametro:	UUID
Nome categoria:	Ricerca
Esito [UUID]	None 1. UUID valido ma non trovato = false [error]. 2. UUID trovato = true [PROPERTY UUID OK].

Parametro:	Tipologia
Nome categoria:	Ricerca
Esito [ETIP]	None 1. Tipologia non trovata = false [error]. 2. Tipologia valida ed esistente = true [PROPERTY ETIP OK].

Test Case ID	Test frame	Esito
TC RF 3_4	UUID 1	**Errato**: Questionario non trovato.
TC RF 3_5	ETIP 1	**Errato**: Tipologia non trovata.
TC RF 3_6	UUID2, ETIP 1	**Errato**: UUID valido ma tipologia non trovata.
TC RF 3_7	UUID2, ETIP 2	**Corretto**: Recupero e parsing avvenuto con successo.

1.3 Unit 3: Validazione Calcolo Score (service)

Obiettivo: Verificare l'integrità degli input per il calcolo.

Parametro:	Tipologia Score
Nome categoria:	Esistenza
Validazione [VALS]	None 1. Nome Tipologia vuoto = false [error]. 2. Tipologia non trovata = false [error]. 3. Tipologia esistente = true [PROPERTY VALSC_OK].

Parametro:	Risposte
Nome categoria:	Struttura
Validazione [VALR]	None 1. Risposte Null o Non Array = false [error]. 2. Array risposte valide = true [PROPERTY VALR_OK].

Test Case ID	Test frame	Esito
TC RF 3_8	VALSC1	**Errato**: Nome tipologia mancante.
TC RF 3_9	VALSC2	**Errato**: Tipologia non trovata.
TC RF 3_10	VAL SC3, VALR1	**Errato**: Risposte non valide.
TC RF 3_11	VAL SC3, VALR2	**Corretto**: Input validi per il calcolo.

1.4 Unit 4: Esecuzione Calcolo Score (service)

Obiettivo: Verifica la correttezza del calcolo matematico.

Parametro:	Risposte
Nome categoria:	Valori
Validazione [VALR]	None 1. Valori non validi = false [error]. 2. Valori validi = true [PROPERTY VALR_OK].

Test Case ID	Test frame	Esito
TC RF 3_12	VAL R1	**Errato**: Valori non validi.
TC RF 3_13	VAL R2	**Corretto**: Valori validi score calcolato.

1.5 Unit 5: Validazione Invio (service)

Obiettivo: Verificare i dati prima del salvataggio.

Parametro:	ID Paziente
Nome categoria:	Esistenza
Validazione [VALPID]	None 1. Nulla vuoto = false [error]. 2. Non trovato = false [error]. 3. Paziente esistente = true [PROPERTY VALPID_OK].

Parametro:	Risposte
Nome categoria:	Completezza
Validazione [VAL RIS]	None 1. Array vuoto = false [error]. 3. Array popolato = true [PROPERTY VALRIS_OK].

Test Case ID	Test frame	Esito
TC_RF3_12	VALPID1	**Errato**: ID Paziente mancante.
TC_RF3_13	VALPID2	**Errato**: Paziente non trovato.
TC_RF3_14	VALPID3, VALRIS2	**Errato**: Nessuna risposta fornita.
TC_RF3_15	VALPID3, VALRIS2	**Errato**: Dati validi e inviati correttamente.

1.6 Unit 6: Esecuzione Invio (service)

Obiettivo: Verifica salvataggio.

Parametro:	Operazione DB
Nome categoria:	Esito Insert
Esito[DB]	None 1. Insert Fallito = false [error]. 2. Insert Riuscito = true [PROPERTY DB_OK].

Test Case ID	Test frame	Esito
TC_RF3_16	DB1	**Errore:** Errore inserimento DB.
TC_RF3_17	DB2	**Corretto:** Questionario inserito e score aggiornato.

1.7 Unit 7: Gestione Richiesta Get (controller)

Obiettivo: Verifica endpoint di recupero.

Parametro:	Chiamata Service
Nome categoria:	Delega
Esito[GET]	None 1. Lancia eccezione = false [error]. 2. Torna dati = true [PROPERTY GET OK].

Test Case ID	Test frame	Esito
TC_RF3_18	GET1	**Errore:** Service fallisce.
TC_RF3_19	GET2	**Corretto:** Ritorna dati.

1.8 Unit 8: Gestione Richiesta Start (controller)

Obiettivo: Verifica endpoint di avvio.

Parametro:	Chiamata Service
Nome categoria:	Delega
Esito[START]	None 1. Lancia eccezione = false [error]. 2. Torna dati = true [PROPERTY START OK].

Test Case ID	Test frame	Esito
TC_RF3_20	START1	**Errore:** Service fallisce.
TC_RF3_21	START2	**Corretto:** Ritorna questionario.

1.9 Unit 9: Gestione richiesta Submit (controller)

Obiettivo: Verifica endpoint di avvio.

Parametro:	Chiamata Service
Nome categoria:	Delega
Esito[SUB]	None 1. Lancia eccezione = false [error]. 2. Torna dati = true [PROPERTY SUB OK].

Test Case ID	Test frame	Esito
TC RF 3_22	SUB1	**Errore:** Service fallisce.
TC_RF3_23	SUB2	**Corretto:** Ritorna dati.

SINTONIA
La mente trova pace, un passo alla volta

2 Inserimento Stato d'Animo (UC_PZT_5)

2.1 Unit 1: Validazione Dati (service)

Obiettivo: Verifica che i dati in input rispettino i vincoli di integrità e formato.

Parametro:	ID Paziente
Nome categoria:	Esistenza
Validazione[VAL PAZ]	None 1. Null o Stringa Vuota = false [error]. 2. Paziente non trovato = false [error]. 3. Paziente trovato = true [PROPERTY VAL PAZ OK]

Parametro:	Umore
Nome categoria:	Valore Umore
Validazione[VALUM]	None 1. Null o Stringa Vuota = false [error]. 2. Tipo non stringa = false [error]. 3. Valore non presente nelle insieme degli umori = false [error]. 4. Valore valido = true [PROPERTY VALUM_OK]

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Intensità
Nome categoria:	Valore Range
Validazione[VALIANT]	None 1. Null o Stringa Vuota = false [error]. 2. Tipo non numerico = false [error]. 3. 1 <= valore <= 10 = false [error]. 4. Valore valido = true [PROPERTY VALINT_OK]

Parametro:	Note
Nome categoria:	Lunghezza
Validazione[VAL NOTE]	None 1. Tipo non stringa = false [error]. 2. Lunghezza > 500 caratteri = false [error]. 3. Valore valido = true [PROPERTY VAL NOTE OK]

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Data Inserimento
Nome categoria:	Formato
Validazione[VALDATA]	None 1. Tipo non stringa = false [error]. 2. Lunghezza > 500 caratteri = false [error]. 3. Valore valido = true [PROPERTY VAL NOTE OK]

SINTONIA
La mente trova pace, un passo alla volta

Test Case ID	Test frame	Esito
TC RF 5_1	VAL PAZ1	**Errore:** ID Paziente mancante.
TC RF 5_2	VAL PAZ 2	**Errore:** Paziente non trovato.
TC RF 5_3	VAL PAZ 3, VALUE 1	**Errore:** Umore mancante.
TC RF 5_4	VAL PAZ,3 VALUE M2	**Errore:** Tipo valore non valido.
TC RF 5_5	VAL PAZ 3, VALUE M3	**Errore:** Umore non valido.
TC RF 5_6	VAL PAZ 3, VALIANT 3	**Errore: Intensità fuori range.**
TC RF 5_7	VALPA Z3, VAL NOTE2	**Errore: Note troppo lunghe.**
TC RF 5_8	VAL PAZ 3, VALDATA2	**Errore: Data non valida.**
TC RF 5_9	VAL PAZ 3, VALUE M4, VALENTIN4, VAL NOTE3, VAL DATA3	**Corretto:** Tutti i dati validi.

2.2 Unit 2: Esecuzione Creazione (service)

Obiettivo: Verificare il corretto inserimento nel DB e l'assegnazione badge.

Parametro:	Operazione DB
Nome categoria:	Esito Insert
Esito[DB]	None 1. Insert Fallito = false [error]. 2. Insert Riuscito = true [PROPERTY DB_OK].

Parametro:	Badge Service
Nome categoria:	Integrazione Badge
Esito[BADGE]	None 1. Assegnazione Badge fallita = false [error]. 2. Assegnazione badge riuscita = true [PROPERTY BADGE OK].

Test Case ID	Test frame	Esito
TC RF 5_10	DB1	**Errore:** Errore inserimento DB.
TC RF 5_11	DB2,BADGE 1	**Errore:** Assegnazione fallita.
TC RF 5_12	DB2, BADGE 2	**Corretto:** Inserimento riuscito.

2.3 Unit 3: Gestione richiesta (controller)

Obiettivo: Verificare la delega al service.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG OK]

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RISP OK]

Test Case ID	Test frame	Esito
TC RF 5_13	DLG1	**Errato**: Service non chiamato correttamente.
TC RF 5_14	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC RF 5_15	DLG2, RSP2	**Corretto**: Service chiamato correttamente, risultato restituito.

3 Inserimento Contenuto Diario Personale (UC_PZT_19)

3.1 Unit 1: Validazione Contenuto Diario (service)

Obiettivo: Verificare che i campi che si vogliano inserire nel diario siano formalmente corretti.

Parametro:	Titolo Pagina (Facoltativo)
Nome categoria	Titolo
Lunghezza [LTD]	None 1. Lunghezza > 64 = false [error]. 2. Lunghezza <= 64 = true [PROPERTY LTD OK].

Parametro:	Contenuto Pagina (Obbligatorio)
Nome categoria	Contenuto
Lunghezza [LCD]	None 1. Lunghezza <= 0 false [error]. 2. Lunghezza => 2000 false [error]. 3. 0 <= Lunghezza <= 2000 = true [PROPERTY LCD OK].

SINTONIA
La mente trova pace, un passo alla volta

Test Case ID	Test frame	Esito
TC RF 19_1	LTD 1	**Errato**: Il titolo supera il numero limite di caratteri.
TC RF 19_2	LTD 2, LCD1	**Errato**: Il campo contenuto non può essere vuoto.
TC RF19 3	LTD 2, LCD2	**Errato**: Il campo contenuto non può superare 2000 caratteri.
TC RF 19 4	LTD 2, LCD3	**Corretto**: I campi inseriti rispettano tutti i vincoli formali.

3.2 Unit 2: creazione pagina diario (service)

Obiettivo: Verificare il flusso di business e l'interazione con il database.

Parametro:	Stato Database
Nome categoria	Operazione Database
Esito[DB]	None 1. Inserimento fallito (nessun ID ritornato) = false [error] 2. Inserimento riuscito (ID ritornato) = true [PROPERTY DB_OK]

Parametro:	Assegnazione Badge
Nome categoria:	Operazione Delega
Esito [BDG]	None 1. Servizio badge non chiamato = false [error] 2. Servizio badge chiamato correttamente = true [PROPERTY BDG OK]

Test Case ID	Test frame	Esito
TC RF 19_5	DB1	**Errato**: Inserimento nel DB fallito.
TC RF 19 6	DB2,BDG 1	**Errato**: Servizio badge non chiamato.
TC RF 19 7	DB2, BDG 2	**Corretto**: Pagina inserita correttamente, servizio badge chiamato.

3.3 Unit 3: Gestione richiesta (control)

Obiettivo: Verificare che il controller estragga i parametri della request e deleghi al servizio.

Parametro:	User ID
Nome categoria:	Validazione ID
Validazione [VALID]	None 1. User ID non presente = false [error] 2. User ID presente = true [PROPERTY VALID_OK]

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG OK]

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RISP OK]

Test Case ID	Test frame	Esito
TC RF 19 8	VALID 1	**Errato**: User ID non trovato.
TC RF 19 9	VALID 2,DLG1	**Errato**: Service non chiamato correttamente.
TC RF 19_10	VALID 2, DLG 2, RSP1	**Errato**: Nessun risultato restituito.
TC RF 19_11	VALID 2, DLG 2, RSP2	**Corretto**: User ID presente, service chiamato correttamente, risultato restituito.

4 Inserimento Domanda Forum (UC_PZT_23)

4.1 Unit 1: Validazione Domanda (service)

Obiettivo: Verificare che i dati in ingresso rispettino i vincoli formali.

Parametro:	Titolo domanda
Nome categoria:	Titolo
Lunghezza [LTI]	None 1. Lunghezza <= 0 = false [error] 2. Lunghezza > 64 = false [error] 3. 0 < Lunghezza <= 64 = true [PROPERTY LTI OK]

Parametro:	Testo del messaggio
Nome categoria:	Contenuto
Lunghezza [LTE]	None 1. Lunghezza <= 0 = false [error] 2. Lunghezza > 0 = true [PROPERTY LTE OK]

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Categoria tematica
Nome categoria:	Scelta per la categoria
Lunghezza [SC]	None 1. La categoria non è stata selezionata (NULL) = false [error] 2. La categoria è stata selezionata = true [PROPERTY SC_OK]
Validità [VAL]	None 1. Categoria non valida (es. inesistente, archiviata o solo lettura) = false [error] 2. Categoria valida (esistente e attiva) = true [PROPERTY VAL OK]

SINTONIA
La mente trova pace, un passo alla volta

Test Case ID	Test frame	Esito
TC_RF23_1	LTI 1	**Errato**: Il Titolo è vuoto
TC RF 23_2	LTI 2	**Errato**: il Titolo è > 64 caratteri
TC RF23 3	LTI 3, LTE 1	**Errato**: Il Testo del messaggio è vuoto.
TC RF23 4	LTI 3, LTE 2, SC1	**Errato**: Nessuna Categoria Tematica selezionata.
TC RF 23_5	LTI 3, LTE2, SC2,VAL1	**Errato**: Categoria non valida.
TC RF23 6	LTI 3, LTE2, SC2,VAL2	**Corretto**: Tutti i dati rispettano i vincoli formali.

4.2 Unit 2: Inserimento Domanda (service)

Obiettivo: Verificare il flusso di business e l'interazione con il database.

Parametro:	Risultato Inserimento Database
Nome categoria:	Operazione Database
Esito [DB]	None 1. Inserimento fallito (nessun ID ritornato) = false [error] 2. Inserimento riuscito (ID ritornato) = true [PROPERTY DB_OK]

Parametro:	Assegnazione Badge
Nome categoria:	Operazione Badge
Esito [BDG]	None 1. Servizio badge non chiamato = false [error] 2. Servizio badge chiamato correttamente = true [PROPERTY BDG OK]

Test Case ID	Test frame	Esito
TC RF23 7	DB1	**Errato**: Inserimento nel DB fallito.
TC RF23 8	DB2,BDG 1	**Errato**: Servizio badge non chiamato.
TC RF23 9	DB2, BDG 2	**Corretto**: Domanda inserita correttamente, servizio badge chiamato.

4.3 Unit 3: Inserimento Domanda (control)

Obiettivo: Verificare che il controller estragga i parametri della request e deleghi al servizio.

Parametro:	User ID
Nome categoria:	Validazione ID
Validazione [VALID]	None 1. User ID non presente = false [error] 2. User ID presente = true [PROPERTY VALID_OK]

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG OK]

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RISP OK]

Test Case ID	Test frame	Esito
TC RF 23_10	VALID 1	**Errato**: User ID non trovato.
TC RF 23_11	VALID 2,DLG1	**Errato**: Service non chiamato correttamente.
TC_RF23_12	VALID 2, DLG 2, RSP1	**Errato**: Nessun risultato restituito.
TC RF 23 13	VALID 2, DLG 2, RSP2	**Corretto**: User ID presente, service chiamato correttamente, risultato restituito.

5 Revisione questionario paziente (UC_PSI_1)

5.1 Unit 1: Validazione revisione (service)

Obiettivo: Verificare che l'operazione di revisione sia permessa.

Parametro:	ID Questionario
Nome categoria:	Formattazione
Validazione [VINO]	None 1. ID Questionario Null o Vuoto = false [error]. 2. ID Questionario inserito = true [PROPERTY VINO _OK].

Parametro:	ID Psicologo
Nome categoria:	Formattazione
Validazione [VINO]	None 1. ID Psicologo Null o Vuoto = false [error]. 2. ID Psicologo inserito = true [PROPERTY VINO OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	ID Questionario
Nome categoria:	Presenza ed Esistenza
Validazione[VEX]	None 1. Questionario non trovato = false [error]. 2. Questionario esistente = true [PROPERTY VEX OK].

Parametro:	Revisionato
Nome categoria:	Stato Revisione
Validazione [VREV]	None 1. Già Revisionato (true) = false [error]. 2. Non ancora revisionato (false) = true [PROPERTY VINO _OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Invalidazione
Nome categoria:	Stato Invalidazione
Validazione [VINV]	None 1. Già invalidato (true) = false [error]. 2. Richiesta invalidazione in corso (di Psicologia Richiedente != null) = false [error]. 3. Nessuna invalidazione o richiesta = true [PROPERTY VINO _OK].

Test Case ID	Test frame	Esito
TC_RF1_1	VIN1	**Errore:** L'ID del questionario non può essere vuoto.
TC_RF1_2	VINO 2, VINO 1	**Errore:** L'ID dello psicologo non può essere vuoto.
TC_RF1_3	VINO 2, VINO 2, VEX 1	**Errore:** Questionario non trovato.
TC_RF1_4	VINO 2, VINO 2, VEX 2, REV1	**Errore:** Non si può revisionare un questionario già revisionato..
TC_RF1_5	VQ2, VIP 2, VEX 2, REV2, VIN 1	**Errore:** Non si può revisionare un questionario che è stato invalidato.
TC_RF1_6	VQ2, VINO 2, VEX 2, REV2, VIN V2	**Errore:** Non si può revisionare un questionario per cui è stata richiesta l'invalidazione.
TC_RF1_6	VQ2, VINO 2, VEX 2, REV2, VINC3	**Corretto:** Tutti i campi rispettano le regole formali

5.2 Unit 2: Esecuzione Revisione Questionario (service)

Obiettivo: Verificare la corretta aggregazione dei dati e l'aggiornamento del campo revisionato nel database.

Parametro:	Aggiornamento revisione
Nome categoria:	Operazione Database
Esito[DB]	None 1. Update fallito = false [error]. 2. Update riuscito = true [PROPERTY DB_OK].

Test Case ID	Test frame	Esito
TC_RF1_7	DB1	**Errore:** Errore durante l'aggiornamento della revisione.
TC_RF1_8	DB2	**Corretto:** Revisione aggiornata correttamente.

5.3 Unit 3: Gestione richiesta (controller)

Obiettivo: Verificare che il controller estragga i parametri della request e deleghi al servizio.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG OK]

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RISP OK]

Test Case ID	Test frame	Esito
TC_RF4_9	DLG1	**Errato**: Service non chiamato correttamente.
TC_RF4_10	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC RF 4_11	DLG2, RSP2	**Corretto**: Service chiamato correttamente, risultato restituito.

5 Revisione questionario paziente (UC_PSI_1)

5.1 Unit 1: Validazione revisione (service)

Obiettivo: Verificare che l'operazione di revisione sia permessa.

Parametro:	ID Questionario
Nome categoria:	Formattazione

Validazione [VINQ]	None
	`1. ID Questionario Null o Vuoto = false [error].`
	`2. ID Questionario inserito = true [PROPERTY VINQ_OK].`

Parametro:	ID Psicologo
Nome categoria:	Formattazione

Validazione [VINP]	None
	`1. ID Psicologo Null o Vuoto = false [error].`
	`2. ID Psicologo inserito = true [PROPERTY VINP_OK].`

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	ID Questionario
Nome categoria:	Presenza ed Esistenza
Validazione[VEX]	None 1. Questionario non trovato = false [error]. 2. Questionario esistente = true [PROPERTY VEX_OK].

Parametro:	Revisionato
Nome categoria:	Stato Revisione
Validazione [VREV]	None 1. Già Revisionato (true) = false [error]. 2. Non ancora revisionato (false) = true [PROPERTY VINQ_OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Invalidazione
Nome categoria:	Stato Invalidazione
Validazione [VINV]	None 1. Già invalidato (true) = false [error]. 2. Richiesta invalidazione in corso (idPsicologoRichiedente != null) = false [error]. 3. Nessuna invalidazione o richiesta = true [PROPERTY VINQ_OK].

Test Case ID	Test frame	Esito
TC_RF1_1	VINQ1	**Errore:** L'ID del questionario non può essere vuoto.
TC_RF1_2	VINQ2, VINP1	**Errore:** L'ID dello psicologo non può essere vuoto.
TC_RF1_3	VINQ2, VINP2, VEX1	**Errore:** Questionario non trovato.
TC_RF1_4	VINQ2, VINP2, VEX2, VREV1	**Errore:** Non si può revisionare un questionario già revisionato..
TC_RF1_5	VINQ2, VINP2, VEX2, VREV2, VINV1	**Errore:** Non si può revisionare un questionario che è stato invalidato.
TC_RF1_6	VINQ2, VINP2, VEX2, VREV2, VINV2	**Errore:** Non si può revisionare un questionario per cui è stata richiesta l'invalidazione.
TC_RF1_6	VINQ2, VINP2, VEX2, VREV2, VINV3	**Corretto:** Tutti i campi rispettano le regole formali

5.2 Unit 2: Esecuzione Revisione Questionario (service)

Obiettivo: Verificare la corretta aggregazione dei dati e l'aggiornamento del campo revisionato nel database.

Parametro:	Aggiornamento revisione
Nome categoria:	Operazione Database
Esito[DB]	None 1. Update fallito = false [error]. 2. Update riuscito = true [PROPERTY DB_OK].

Test Case ID	Test frame	Esito
TC_RF1_7	DB1	**Errore:** Errore durante l'aggiornamento della revisione.
TC_RF1_8	DB2	**Corretto:** Revisione aggiornata correttamente.

5.3 Unit 3: Gestione richiesta (controller)

Obiettivo: Verificare che il controller estragga i parametri dalla request e deleghi al servizio.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG_OK]

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RSP_OK]

Test Case ID	Test frame	Esito
TC_RF4_9	DLG1	**Errato**: Service non chiamato correttamente.
TC_RF4_10	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC_RF4_11	DLG2, RSP2	**Corretto**: Service chiamato correttamente, risultato restituito.

6 Visualizzazione Alert Clinico (UC_PSI_3)

6.1 Unit 1: Esecuzione Visualizzazione Alert(service)

Obiettivo: Verificare la corretta visualizzazione degli alert clinici.

Parametro:	Aggiornamento questionario
Nome categoria:	Operazione Database
Esito[DB]	None 1. Lettura fallita = false [error]. 2. Lettura riuscita = true [PROPERTY DB_OK].

Test Case ID	Test frame	Esito
TC_RF3_1	DB1	**Errore:** Errore durante la lettura dei dati.
TC_RF3_2	DB2	**Corretto:** Lettura dei dati avvenuta correttamente.

6.2 Unit 2: Gestione visualizzazione (controller)

Obiettivo: Verificare che il controller estragga i parametri dalla request e deleghi al servizio.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG_OK]

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RSP_OK]

Test Case ID	Test frame	Esito
TC_RF3_3	DLG1	**Errato**: Service non chiamato correttamente.
TC_RF3_4	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC_RF21_5	DLG2, RSP2	**Corretto**: Service chiamato correttamente, risultato restituito.

SINTONIA
La mente trova pace, un passo alla volta

7 Terminazione Cura Paziente(UC_PSI_4)

7.1 Unit 1: Validazione terminazione cura (service)

Obiettivo: Verificare che la richiesta di terminazione della cura sia legittima e coerente.

Parametro:	ID Paziente
Nome categoria:	Presenza ed Esistenza
Validazione [VPZT]	None 1. Null o Stringa Vuota = false [error]. 2. ID valido ma Paziente non trovato = false [error]. 3. Paziente esistente = true [PROPERTY VPZT_OK].

Parametro:	ID Psicologo
Nome categoria:	Associazione con paziente
Validazione [VAUTH]	None 1. Null o Stringa Vuota = false [error]. 2. Paziente non assegnato allo psicologo = false [error]. 3. Psicologo assegnato correttamente = true [PROPERTY VAUTH_OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Stato Corrente
Nome categoria:	Stato Attivo
Validazione [VSTATO]	None 1. Paziente già terminato = false [error] 2. Paziente attivo = true [PROPERTY VSTATO_OK].

Test Case ID	Test frame	Esito
TC_RF4_1	VPZT1	**Errore:** l'ID del paziente non può essere vuoto.
TC_RF4_2	VPZT2	**Errore:** Il paziente non è stato trovato.
TC_RF4_3	VPZT3, VAUTH1	**Errore:** l'ID dello psicologo non può essere vuoto .
TC_RF4_4	VPZT3, VAUTH2	**Errore:** Il paziente deve essere assegnato allo psicologo.
TC_RF4_5	VPZT3, VAUTH3, VSTATO1	**Errore:** La cura del paziente è già stata terminata.
TC_RF4_6	VPZT3, VAUTH3, VSTATO2	**Corretto:** I campi inseriti rispettano tutti i vincoli formali.

7.2 Unit 2: Esecuzione terminazione cura (service)

Obiettivo: Verificare l'aggiornamento dello stato e l'assegnazione del prossimo paziente.

Parametro:	Stato Database
Nome categoria:	Operazione Database
Esito [DB]	None 1. Update fallito = false [error] 2. Update riuscito = true [PROPERTY DB_OK].

Parametro:	Assegnazione Automatica
Nome categoria:	Operazione Delega
Esito [DEL]	None 1. Chiamata al servizio assegnazione fallita = false [error]. 3. Servizio assegnazione chiamato correttamente = true [PROPERTY DEL_OK].

Test Case ID	Test frame	Esito
TC_RF4_7	DB1	**Errato**: Update del DB fallito.
TC_RF4_8	DB2, DEL1	**Errato**: Servizio assegnazione non chiamato correttamente.
TC_RF4_9	DB2, DEL2	**Corretto**: Update avvenuto con successo e servizio assegnazione chiamato con successo..

7.3 Unit 3: Gestione richiesta (controller)

Obiettivo: Verificare che il controller estragga i parametri dalla request e deleghi al servizio.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG_OK]

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RSP_OK]

Test Case ID	Test frame	Esito
TC_RF4_10	DLG1	**Errato**: Service non chiamato correttamente.
TC_RF4_11	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC_RF4_12	DLG2, RSP2	**Corretto**: User ID presente, service chiamato correttamente, risultato restituito.

8 Generazione Report(UC_PSI_5)

8.1 Unit 1: Gestione richiesta (service)

Obiettivo: Verificare che la richiesta di generazione report sia valida.

Parametro:	ID Paziente
Nome categoria:	Presenza ed Esistenza
Validazione[VALPZ]	None 1. Null o Stringa Vuota = false [error]. 2. ID valido ma Paziente non trovato = false [error]. 3. Paziente esistente= true [PROPERTY VALPZT_OK].

Test Case ID	Test frame	Esito
TC_RF5_1	VALPZ1	**Errore:** l'ID del paziente non può essere vuoto.
TC_RF5_2	VALPZ2	**Errore:** Il paziente non è stato trovato.
TC_RF5_3	VALPZ3	**Corretto:** I campi inseriti rispettano tutti i vincoli formali.

8.2 Unit 2: Esecuzione Generazione Report (service)

Obiettivo: Verificare la corretta aggregazione dei dati e il salvataggio del report nel database.

Parametro:	Recupero Dati
Nome categoria:	Disponibilità Dati Esito
Esito[DATA]	None 1. Nessun dato presente = false [error]. 2. Dati presenti = true [PROPERTY DATA_OK].

Parametro:	Salvataggio Report
Nome categoria:	Operazione Database
Esito[DB]	None 1. Inserimento fallito = false [error]. 2. Inserimento riuscito = true [PROPERTY DB_OK].

Test Case ID	Test frame	Esito
TC_RF5_4	DATA1	**Errore:** Il report non può essere generato vuoto.
TC_RF5_5	DATA2, DB1	**Errore:** Errore durante il salvataggio del report.
TC_RF5_6	DATA2, DB2	**Corretto:** Report generato e salvato correttamente.

8.3 Unit 3: Gestione richiesta (controller)

Obiettivo: Verificare che il controller estragga i parametri dalla request e deleghi al servizio.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG_OK]

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RSP_OK]

Test Case ID	Test frame	Esito
TC_RF5_7	DLG1	**Errato**: Service non chiamato correttamente.
TC_RF5_8	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC_RF5_9	DLG2, RSP2	**Corretto**: Service chiamato correttamente, risultato restituito.

9 Modifica Priorità Paziente(RF_AS_10)

9.1 Unit 1: Validazione nuova priorità (service)

Obiettivo: Verificare che i dati di input siano formalmente corretti e coerenti con i dati presenti nel database (esistenza paziente e priorità).

Parametro:	Valore priorità
Nome categoria:	Validità lessicale
Validazione [VALP]	None 1. Null o Stringa Vuota = false [error]. 2. Valore non incluso in ['Urgente', 'Breve', 'Differibile', 'Programmabile'] = false [error]. 3. Valore valido = true [PROPERTY VALP_OK].

Parametro:	ID Paziente
Nome categoria:	Presenza ed Esistenza
Validazione[VALPZT]	None 1. Null o Stringa Vuota = false [error]. 2. ID valido ma Paziente non trovato = false [error]. 3. Paziente esistente= true [PROPERTY VALPZT_OK].

Test Case ID	Test frame	Esito
TC_RF10_1	VALP1	**Errore:** La priorità non può essere vuota.
TC_RF10_2	VALP2	**Errore:** La priorità deve essere compresa tra le priorità ammissibili.
TC_RF10_3	VALP3, VALPZT1	**Errore:** l'ID del paziente non può essere vuoto.
TC_RF10_4	VALP3, VALPZT2	**Errore:** Il paziente non è stato trovato.
TC_RF4_5	VALP3, VALPZT3	**Corretto:** I campi inseriti rispettano tutti i vincoli formali.

9.2 Unit 2: Esecuzione modifica priorità (service)

Obiettivo: Verificare l'aggiornamento dello stato nel database.

Parametro:	Stato Database
Nome categoria:	Operazione Database
Esito [DB]	None 1. Update fallito = false [error] 2. Update riuscito = true [PROPERTY DB_OK].

Test Case ID	Test frame	Esito
TC_RF10_6	DB1	**Errato**: Update del DB fallito.
TC_RF10_7	DB2	**Corretto**: Update avvenuto correttamente.

9.3 Unit 3: Gestione richiesta (controller)

Obiettivo: Verificare che il controller estragga i parametri dalla request e deleghi al servizio.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG_OK]

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RSP_OK]

Test Case ID	Test frame	Esito
TC_RF10_8	DLG1	**Errato**: Service non chiamato correttamente.
TC_RF10_9	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC_RF11_10	DLG2, RSP2	**Corretto**: Service chiamato correttamente, risultato restituito.

SINTONIA
La mente trova pace, un passo alla volta

10 Modifica Assegnazione Psicologo (UC_AS_11)

10.1 Unit 1: Validazione assegnazione (service)

Obiettivo: Verificare che i campi per la nuova assegnazione siano formalmente corretti.

Parametro:	ID Paziente (Facoltativo)
Nome categoria	Presenza ed Esistenza
Validazione [VPZT]	None 1. Null o Stringa Vuota = false [error]. 2. Id valido ma Paziente non trovato = false [error]. 3. Paziente esistente = true [PROPERTY VPZT_OK].

Parametro:	ID Psicologo (Obbligatorio)
Nome categoria	Presenza ed Esistenza
Vali dazione [VPSI]	None 1. Null o Stringa Vuota = false [error]. 2. Id valido ma Psicologo non trovato = false [error]. 3. Paziente esistente = true [PROPERTY VPSI_OK].

Test Case ID	Test frame	Esito
TC_RF11_1	VPZT1	**Errato**: L'Id del paziente non può essere nullo.
TC_RF11_2	VPZT2	**Errato**: L'Id fornito deve corrispondere con un paziente esistente.
TC_RF11_3	VPZT3, VPSI1	**Errato**: L'Id dello psicologo non può essere nullo.
TC_RF11_4	VPZT3, VPSI2	**Errato**: L'Id fornito deve corrispondere ad uno psicologo esistente.
TC_RF11_5	VPZT3, VPSI3	**Corretto:** I campi inseriti rispettano tutti i vincoli formali.

10.2 Unit 2: Esecuzione assegnazione (service)

Obiettivo: Verificare che l'aggiornamento avvenga e che venga eseguita la logica di delega.

Parametro:	Stato Database
Nome categoria	Operazione Database
Esito[DB]	None 1. Update fallito = false [error] 2. Update riuscito = true [PROPERTY DB_OK]

Parametro:	Assegnazione Paziente
Nome categoria:	Operazione Delega
Esito [DEL]	None 1. Chiamata al servizio assegnazione fallita = false [error] 2. Servizio assegnazione non chiamato (Il nuovo psicologo ha lo stesso id di quello precedente) = true [PROPERTY DEL_SKIP] 3. Servizio assegnazione chiamato correttamente = true [PROPERTY DEL_OK]

Test Case ID	Test frame	Esito
TC_RF11_6	DB1	**Errato**: Update del DB fallito.
TC_RF11_7	DB2, DEL1	**Errato**: Servizio assegnazione non chiamato correttamente.
TC_RF11_8	DB2, DEL2	**Corretto**: Update avvenuto con successo e servizio assegnazione non chiamato perché lo psicologo è lo stesso.
TC_RF11_9	DB2, DEL3	**Corretto**: Update avvenuto con successo e servizio assegnazione chiamato correttamente.

10.3 Unit 3: Gestione richiesta (controller)

Obiettivo: Verificare che il controller estragga i parametri dalla request e deleghi al servizio.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer
Delega [DLG]	None 1. Service chiamato con parametri errati = false [error] 2. Service chiamato con parametri corretti = true [PROPERTY DLG_OK]

Parametro:	Risposta
Nome categoria:	Response Handling
Risposta [RSP]	None 1. Controller non restituisce il risultato = false [error] 2. Controller restituisce il risultato = true [PROPERTY RSP_OK]

Test Case ID	Test frame	Esito
TC_RF19_10	DLG1	**Errato**: Service non chiamato correttamente.
TC_RF19_11	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC_RF19_12	DLG2, RSP2	**Corretto**: User ID presente, service chiamato correttamente, risultato restituito.

11 Accettazione richiesta invalidazione (UC_AS_21)

11.1 Unit 1: Validazione richiesta (service)

Obiettivo: Verificare che l'operazione di invalidazione sia permessa.

Parametro:	ID Questionario
Nome categoria:	Formattazione
Validazione [VALQ]	None 1. ID Questionario Null o Vuoto = false [error]. 2. ID Questionario inserito = true [PROPERTY VALQ_OK].

Parametro:	Email Amministratore
Nome categoria:	Formattazione
Validazione [VALA]	None 1. Email Amministratore Null o Vuoto = false [error]. 2. Email Amministratore inserito = true [PROPERTY VALA_OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	ID Questionario
Nome categoria:	Presenza ed Esistenza
Validazione [VALPQ]	None 1. Questionario non trovato = false [error]. 2. Questionario esistente = true [PROPERTY VALPQ_OK].

Parametro:	Email Amministratore
Nome categoria:	Presenza ed Esistenza
Validazione [VALPA]	None 1. Amministratore non trovato = false [error]. 2. Amministratore esistente = true [PROPERTY VALPQ_OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Invalidazione
Nome categoria:	Stato Invalidazione
Validazione [VINV]	None 1. Già Invalidato (true) = false [error]. 2. Non ancora invalidato (false) = true [PROPERTY VINV_OK].

Test Case ID	Test frame	Esito
TC_RF21_1	VALQ1	**Errore:** L'ID del questionario non può essere vuoto.
TC_RF21_2	VALQ2, VALA1	**Errore:** L'Emal dell'amministratore non può essere vuota.
TC_RF21_3	VALQ2, VALA2, VALPQ1	**Errore:** Questionario non trovato.
TC_RF21_4	VALQ2, VALA2, VALPQ2, VALPA1	**Errore:** Amministratore non trovato.
TC_RF21_5	VALQ2, VALA2, VALPQ2, VALPA2, VINV1	**Errore:** Non si può invalidare un questionario che è già stato invalidato.
TC_RF21_6	VALQ2, VALA2, VALPQ2, VALPA2, VINV2	**Corretto:** Tutti i campi rispettano le regole formali

11.2 Unit 2: Ricerca precedente questionario con cambiamento (service)

Obiettivo: Verificare la corretta ricerca di un questionario precedente con il campo cambiamento vero.

Parametro:	ID Questionario
Nome categoria:	Formattazione
Validazione [VALPZT]	None 1. ID Paziente Null o Vuoto = false [error]. 2. ID Paziente inserito = true [PROPERTY VALPZT_OK].

Parametro:	Data Questionario
Nome categoria:	Formattazione
Validazione [VALD]	None 1. Data questionario Null o Vuota = false [error]. 1. Data questionario > Current Date = false [error]. 2. Data questionario inserita correttamente = true [PROPERTY VALD_OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	ID Paziente
Nome categoria:	Presenza ed Esistenza
Validazione [VALPP]	None 1. Paziente non trovato = false [error]. 2. Paziente esistente = true [PROPERTY VALPP_OK].

Parametro:	Questionario
Nome categoria:	Operazione Database
Esito[FIND]	None 1. Nessun questionario con cambiamento(true) precedente trovato == true (null) [PROPERTY FIND_NULL]. 1. Questionario con cambiamento(true) precedente trovato == true (obj) [PROPERTY FIND_NULL].

Test Case ID	Test frame	Esito
TC_RF21_7	VALPZT1	**Errore:** L'id del paziente non è inserito.
TC_RF21_8	VALPZT2, VALD1	**Errore:** La data del questionario invalidato non è inserita.
TC_RF21_9	VALPZT2, VALD2,	**Errore:** La data del questionario invalidato non è valida.
TC_RF21_10	VALPZT2, VALD3, VALPP1	**Errore:** Il paziente non esiste.
TC_RF21_11	VALPZT2, VALD3, VALPP2, FIND1	**Corretto:** Non è stato trovato nessun questionario precedente a quello fornito con campo cambiamento = true.
TC_RF21_12	VALPZT2, VALD3, VALPP2, FIND2	**Corretto:** Questionario con campo cambiamento = true trovato e restituito.

11.3 Unit 3: Ricalcolo score (service)

Obiettivo: Verificare il corretto ricalcolo dello score del paziente.

Parametro:	ID Paziente
Nome categoria:	Formattazione
Validazione [VALPZT]	None 1. ID Paziente Null o Vuoto = false [error]. 2. ID Paziente inserito = true [PROPERTY VALPZT_OK].

Parametro:	Data Questionario
Nome categoria:	Formattazione
Validazione [VALD]	None 1. Data questionario Null o Vuota = false [error]. 1. Data questionario > Current Date = false [error]. 2. Data questionario inserita correttamente = true [PROPERTY VALD_OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	ID Paziente
Nome categoria:	Presenza ed Esistenza
Validazione [VALPP]	None 1. Paziente non trovato = false [error]. 2. Paziente esistente = true [PROPERTY VALPP_OK].

Parametro:	Score
Nome categoria:	Calcolo dello score
Calcolo[CALCS]	None 1. Non raggiunge mai la soglia (fascia persa) = true (false) [PROPERTY TREND_DOWN] 2. Raggiunge la soglia partendo da sotto (nuovo cambiamento) = true (true) [PROPERTY TREND_UP] 3. Raggiungeva già la soglia (nessun cambiamento) = true (true) [PROPERTY TREND_STABLE]

Test Case ID	Test frame	Esito
TC_RF21_13	VALPZT1	**Errore:** L'id del paziente non è inserito.
TC_RF21_14	VALPZT2, VALD1	**Errore:** La data del questionario invalidato non è inserita.
TC_RF21_15	VALPZT2, VALD2,	**Errore:** La data del questionario invalidato non è valida.
TC_RF21_16	VALPZT2, VALD3, VALPP1	**Errore:** Il paziente non esiste.
TC_RF21_17	VALPZT2, VALD3, VALPP2, CALCS1	**Corretto:** Lo score viene ricalcolato e non raggiunge la fascia precedente alla invalidazione.
TC_RF21_18	VALPZT2, VALD3, VALPP2, CALCS2	**Corretto:** Lo score viene ricalcolato e raggiunge una nuova fascia.
TC_RF21_19	VALPZT2, VALD3, VALPP2, CALCS3	**Corretto:** Lo score viene ricalcolato e resta nella stessa fascia precedente all'invalidazione.

11.4 Unit 4: Downgrade priorità (service)

Obiettivo: verificare la corretta assegnazione della priorità.

Parametro:	ID Paziente
Nome categoria:	Formattazione
Validazione [VALPZT]	None 1. ID Paziente Null o Vuoto = false [error]. 2. ID Paziente inserito = true [PROPERTY VALPZT_OK].

Parametro:	ID Paziente
Nome categoria:	Presenza ed Esistenza
Validazione [VALPP]	None 1. Paziente non trovato = false [error]. 2. Paziente esistente = true [PROPERTY VALPP_OK].

SINTONIA
La mente trova pace, un passo alla volta

Parametro:	Priorità
Nome categoria:	Assegnazione Priorità
Assegnazione [ASP]	None 1. Urgente -> Breve = true [PROPERTY U_TO_B] 2. Breve -> Differibile = true [PROPERTY B_TO_D] 3. Differibile -> Programmabile = true [PROPERTY D_TO_P] 4. Programmabile -> Programmabile = true [PROPERTY P_TO_P]

Test Case ID	Test frame	Esito
TC_RF21_20	VALPZT1	**Errore:** L'id del paziente non è inserito.
TC_RF21_21	VALPZT2, VALPP1	**Errore:** Il paziente non esiste.
TC_RF21_22	VALPZT2, VALPP2, ASP1	**Corretto:** La priorità è scesa da Urgente a Breve.
TC_RF21_23	VALPZT2, VALPP2, ASP2	**Corretto:** La priorità è scesa da Breve a Differibile.
TC_RF21_24	VALPZT2, VALPP2, ASP3	**Corretto:** La priorità è scesa da Differibile a Programmabile.
TC_RF21_25	VALPZT2, VALPP2, ASP4	**Corretto:** La priorità è rimasta a programmabile.

11.5 Unit 5: Esecuzione Invalidazione Questionario (service)

Obiettivo: Verificare la corretta aggregazione dei dati e l'aggiornamento del campo revisionato nel database.

Parametro:	Aggiornamento questionario
Nome categoria:	Operazione Database
Esito[DB]	None 1. Update fallito = false [error]. 2. Update riuscito = true [PROPERTY DB_OK].

Test Case ID	Test frame	Esito
TC_RF21_26	DB1	**Errore:** Errore durante l'aggiornamento della revisione.
TC_RF21_27	DB2	**Corretto:** Revisione aggiornata correttamente.

11.6 Unit 6: Gestione richiesta (controller)

Obiettivo: Verificare che il controller estragga i parametri dalla request e deleghi al servizio.

Parametro:	Delega Service
Nome categoria:	Chiamata Service Layer

Delega [DLG]	None
	`1. Service chiamato con parametri errati = false [error]`
	`2. Service chiamato con parametri corretti = true [PROPERTY DLG_OK]`

Parametro:	Risposta
Nome categoria:	Response Handling

Risposta [RSP]	None
	`1. Controller non restituisce il risultato = false [error]`
	`2. Controller restituisce il risultato = true [PROPERTY RSP_OK]`

Test Case ID	Test frame	Esito
TC_RF21_28	DLG1	**Errato**: Service non chiamato correttamente.
TC_RF21_29	DLG2, RSP1	**Errato**: Nessun risultato restituito.
TC_RF21_30	DLG2, RSP2	**Corretto**: Service chiamato correttamente, risultato restituito.